Bitcoin Double-Spend Website

CSCE A470 Design and Requirements Document

Capstone team project

2/4/2026

1. Overview

The Bitcoin Double-Spend website is a blockchain explorer that allows the user to search any transaction hash that exists on the Bitcoin Blockchain and view transaction details such as the sender and receiver addresses, as well as how much money was moved in the transaction. Websites like Blockchain.com or Blockchair.com are two popular choices for exploring the Bitcoin blockchain. These explorers, among others, do not have the capabilities to analyze the double-spend risk. Our project offers the ability to display real-time double-spending probabilities for any given transaction. These probabilities are based on equations referenced in the Bitcoin whitepaper [1], which use a block-based model, and are compared with time-based probability equations presented in [2]. These probabilistic models form the foundation for our project and help demonstrate that double-spend attacks depend on factors such as network activity, time, and the number of transaction confirmations.

1.1 What is a double-spend attack?

A double-spend attack is a scenario in which an attacker exploits the functionality of the network's consensus mechanisms if conflicting transactions were to occur. The way the network handles conflicts is that if a separate chain of blocks is created that conflicts with the main chain, the network always chooses the longer of the two chains because it has the most cumulative proof of work, i.e., the most computational effort. So an attacker can exploit this by creating a conflicting chain with the transaction they want to alter and attempting to gain acceptance from the network by creating a longer chain. In the figure below, we can see this visualized as the attacker's chain in red has become longer than the green main chain, and will be accepted by the network, causing the main chain to be orphaned.

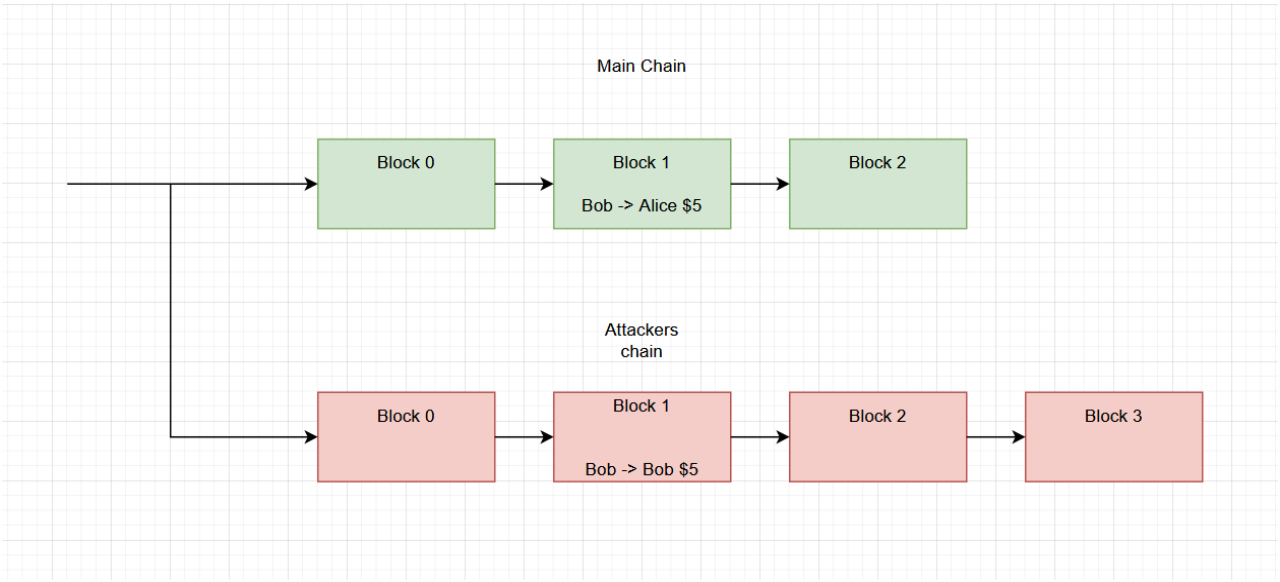


Fig 1: Illustration of Blockchain Double Spend

To calculate the double-spend probability, we use the formula presented in [2], shown below.

$$p_{\text{Double Spend}} = \sum_{i=-N}^{0} p_i^* \left(\frac{\alpha}{1-\alpha} \right)^{-i+1} + 1 - \sum_{i=-N}^{0} p_i^*$$

Fig 2: Double-Spend Equation

In the above equation, the left summation is the probability of the attacker catching up sometime after the transaction has already been accepted, and the right summation is the probability of immediately being able to double-spend after acceptance.

1.2 Why is double-spending bad?

Double-spending is a fundamental threat to the integrity and trustworthiness of the Bitcoin blockchain. Bitcoin is designed to function as a decentralized currency, a peer-to-peer version of electronic money without the need for a financial institution. Once a transaction is complete, it should be unique and irreversible once confirmed by the blockchain. If double-spending occurs, a user would be able to spend the same Bitcoin numerous times, allowing attackers to commit fraudulent exchanges, or potentially reverse transactions after goods or services have been delivered. If this occurred, it would be detrimental to the confidence and integrity of Bitcoin as a reliable payment system, reducing its efficacy as a store of value and medium of exchange.

Although the Bitcoin network is highly resistant to the risk of double-spend attacks, monitoring and analyzing the likelihood of double-spend attacks provides students, Bitcoin/cryptocurrency enthusiasts, merchants, and users transacting Bitcoin with greater confidence that a double-spend has not occurred when sending or receiving transactions.

2. Requirements

The requirements of this project is to give the user the ability to search the entire Bitcoin blockchain spanning from the first Bitcoin transaction in 2008 all the way to the latest bitcoin transactions being added to the Bitcoin network in real time. Once the user searches for a transaction with a given transaction hash (TXID), they will be presented with information regarding that transaction. We will break up the data shown into two parts: the transaction data and the double-spend probabilities. The transaction data that is planned to be shown is the Block # and ID, the time the transaction entered the mempool and block, all input and output amounts and their associated public addresses, and mempool activity data. Alongside the transaction data, we will also show double-spend probabilities that are updated in real time.

2.1 GUI Sketch

The following figure shows a simplified sketch of the double-spend website.

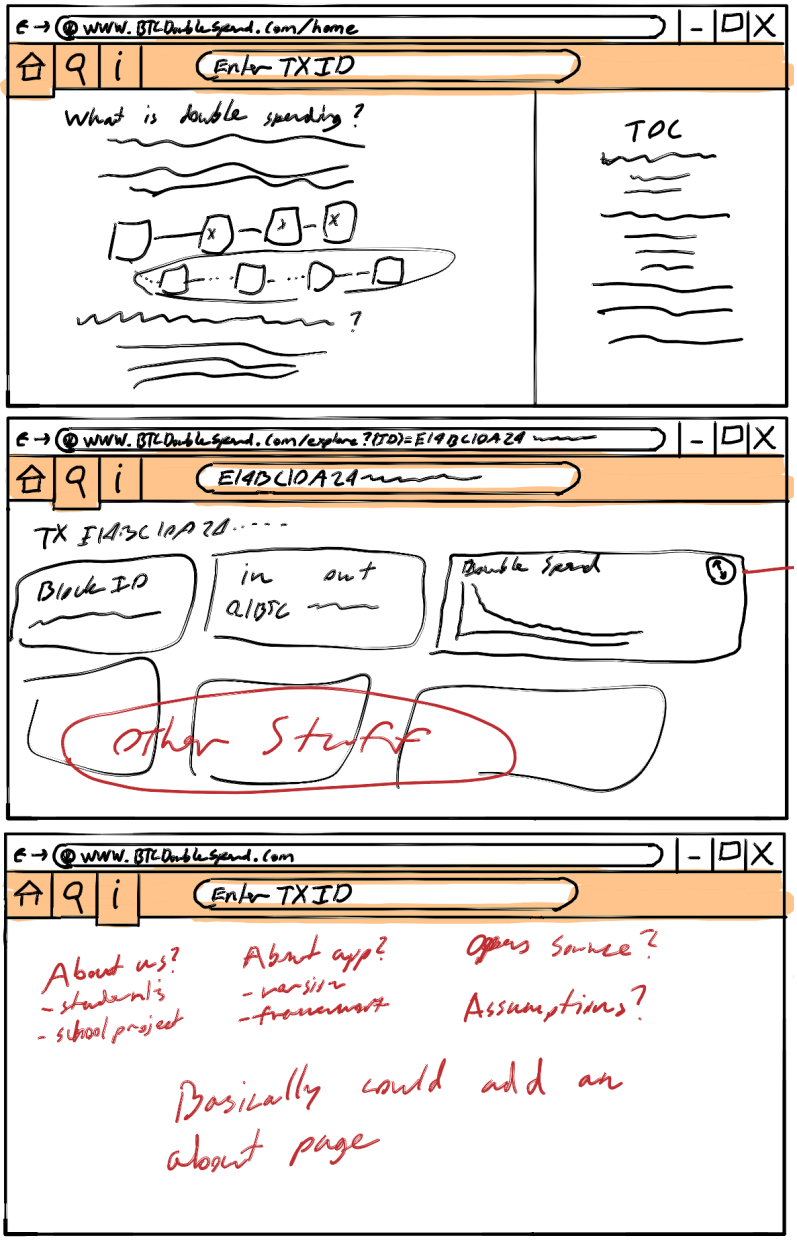


Fig 3: GUI Mockup Sketch

2.2 AI Mockup

The following figures are UI mockups, created with AI tools, of what the website may look like.

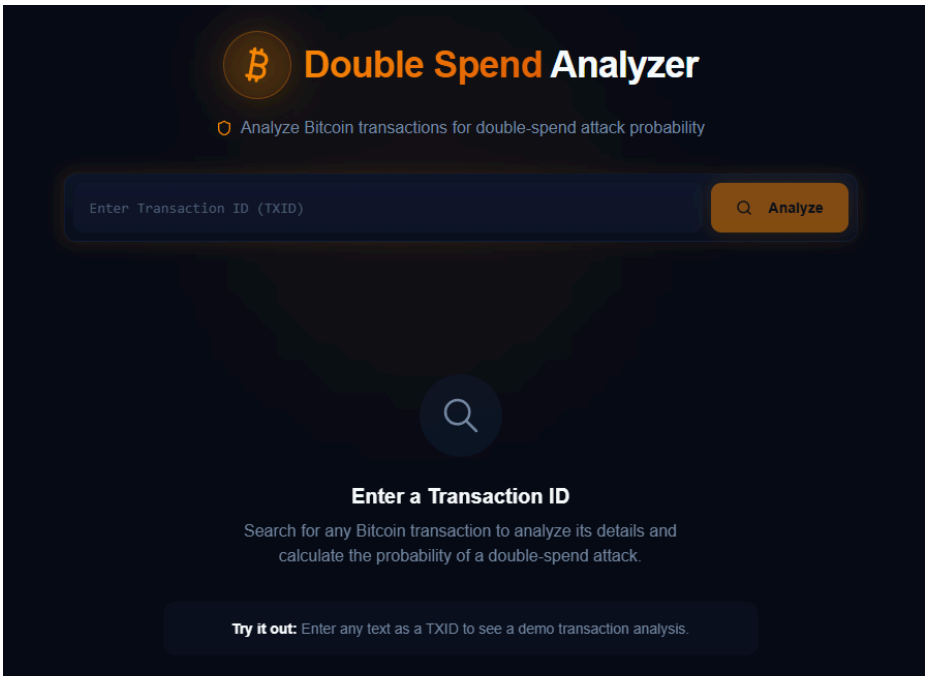


Fig 4: Home page

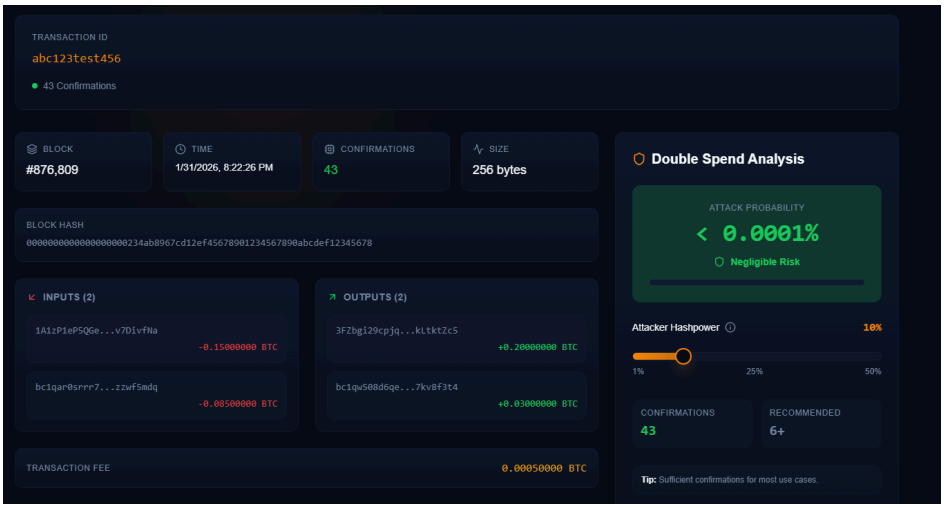


Fig 5: Transaction information page

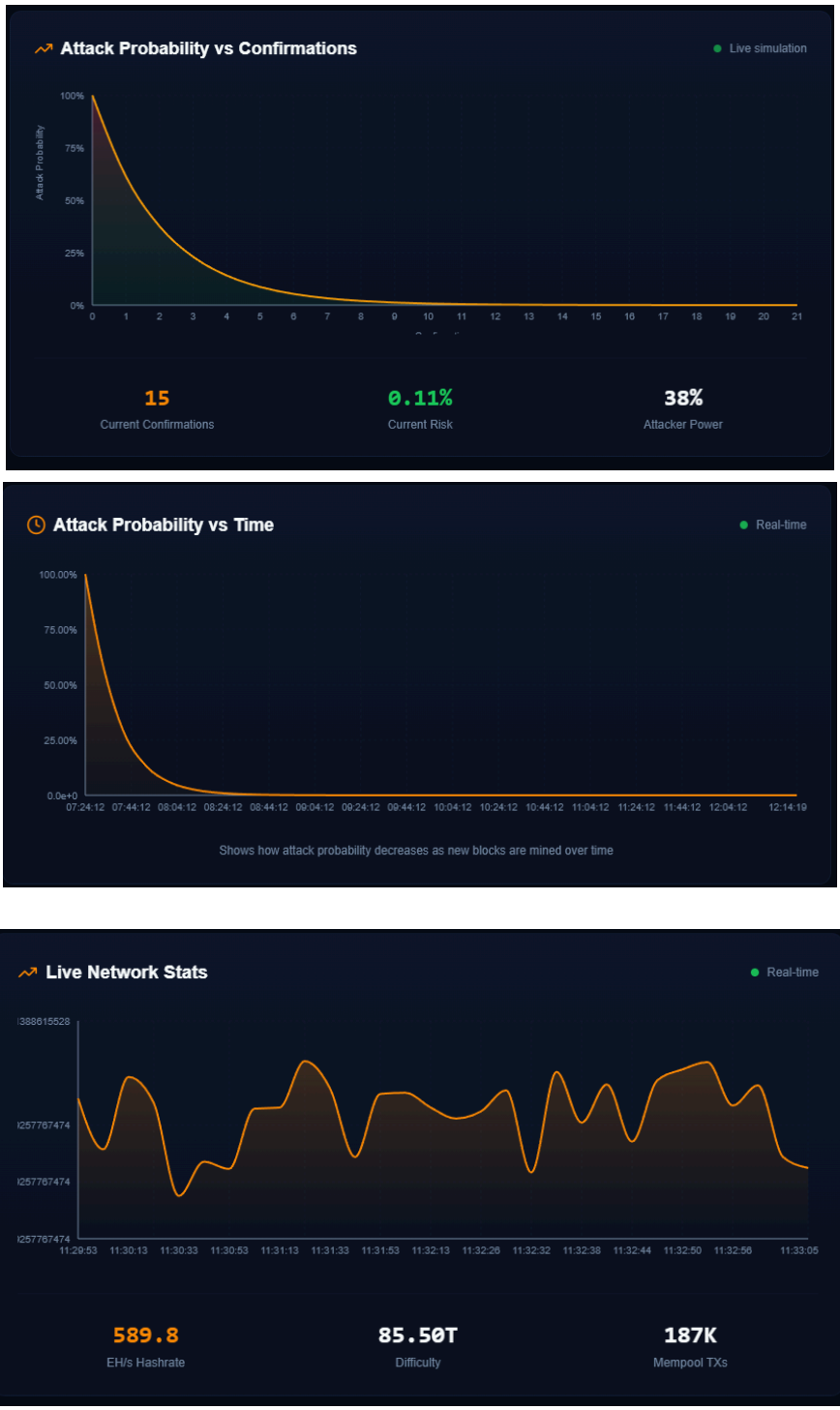


Fig 6-8: Mockup of informational graphs

2.3 User Stories

US1: As a user, I want to search by a transaction ID (TXID), so I can quickly locate on-chain transactions.

US2: As a user, I want a transaction detail page with inputs/outputs/fee/size/confirmations, so I can verify payment details.

US3: As a merchant, I want a double-spend probability score, so I can justify a decision.

US4: As a user, I want to see a double-spend probability graph over time, so I can understand the risk visually.

US5: As a user, I want to have access to double-spend information, so I can be more informed about the risks of Bitcoin transactions

2.4 Sequence Diagram - Search Flow

The following figure is a sequence diagram that represents the flow of a user searching for a transaction with a 256-bit hash (TXID). The actor will enter the TXID in the search bar, then the interface will validate the hash by checking its length and valid characters. Then the interface will send a GET request to the API, which will send a request to the Python BTC service. The Python service will send an RPC request to the BTC Node and receive a JSON response containing transaction details. The Python worker will calculate the double-spend probabilities and send the transaction details with the probabilities to the API. Then the API will

send the data to the UI to display to the actor. Finally, the transaction details will be dynamically refreshed every X seconds.

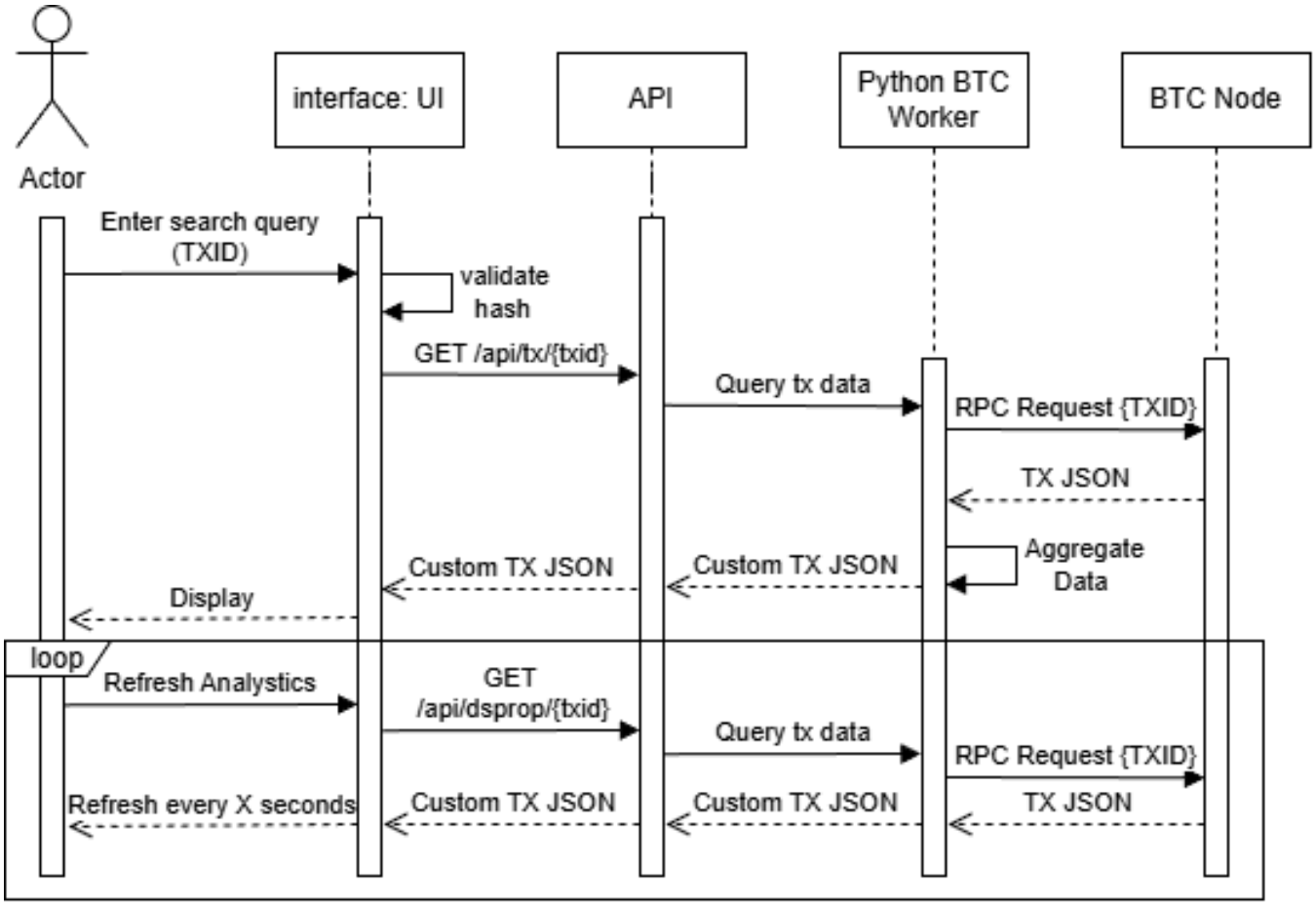


Fig 9: Sequence Diagram

3. Design

The design of the double-spend website consists of 4 major components: the web interface, web API, Python Bitcoin worker service, and the Bitcoin Core Node. The web app will follow Clean Architecture. Clean Architecture is a software design pattern that organizes code into concentric layers to separate concerns, with business logic at the center [3]. The code is

strongly organized in layers to align with Clean Architecture. For example, both the Blazor and Web API solutions have 3 projects each within the solutions. Each project is responsible for a different layer. Additionally, the BTC Core Node and Python Worker service are separate from the API. The following sections describe each major component.

3.1 Backend - Bitcoin Node

The backend of the web application is split into 3 parts: the Web API, the Python worker service, and the Bitcoin Node. To gain access to the required data, access to a blockchain node is needed. Bitcoin Core is the reference software implementation of the Bitcoin network. When Bitcoin Core is downloaded and run on its host machine, it becomes part of the Bitcoin network by maintaining a copy of the entire Bitcoin Blockchain as well as validating and relaying transactions to other nodes in the network. This can be used to query any blockchain transaction spanning all the way back to 2008 and transactions being added to the network in real time.

3.2 Backend - Python worker service

The Python worker service will be responsible for communicating directly with the blockchain node and data processing. The Python worker will receive a transaction hash request from the web API, query the node, and receive a JSON of the data associated with the transaction. The data processing will involve cleaning up raw data to only include required information and calculating the double-spend probabilities. After the data is processed, it will be packaged into a JSON and sent to the API. The Python service will continue to refresh the double-spend probability at an interval.

3.3 Backend - ASP.NET Core Web API

The API, written in C#, acts as a middleware between the services and the user interface. The API is responsible for sending and receiving external resources, converting data as needed, and relaying the data to the user interface.

3.4 Frontend - Blazor Web UI

The project is using a Blazor Web UI for the interface. The interface is solely responsible for displaying information to the user. There should be as little business logic inside the UI code. Blazor consists of C# and HTML via Razor files. The code will follow Clean Architecture.

3.5 High-Level Architecture Diagram

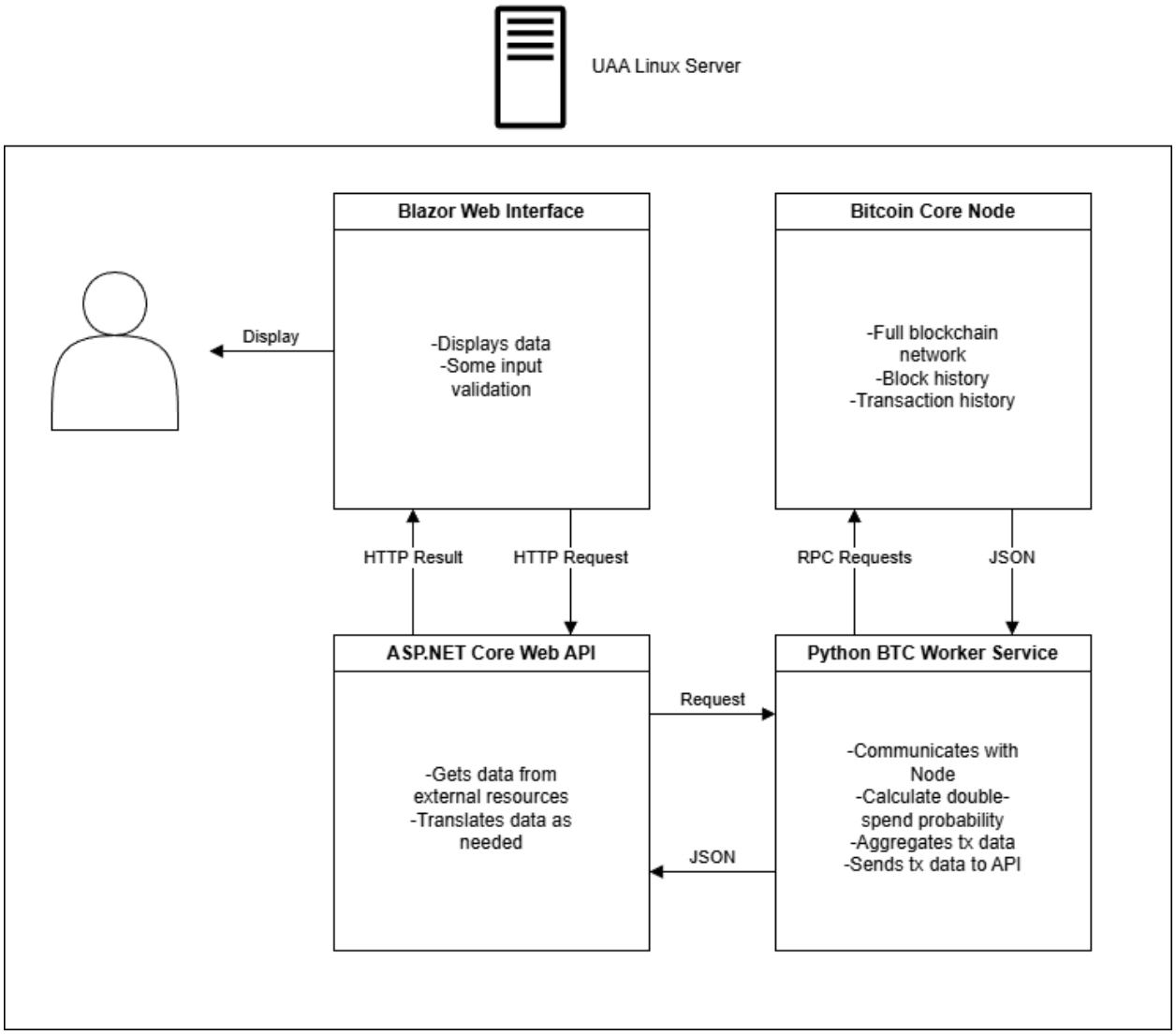


Fig 10: High-Level Architecture Diagram

4. Engineering Standards and Project Constraints

This project follows the established Bitcoin protocol standards through the Bitcoin Core implementation, ensuring appropriate adherence to network rules and transaction procedures.

The system relies on SHA-256 cryptographic hashing and ECDSA digital signatures for transaction verification. Communication between system components follows the RESTful API design using HTTPS protocols.

The project is constrained by performance requirements associated with real-time calculations regarding the probability of a double-spend attack, as well as the blockchain data retrieval latency. Hardware and storage limitations are also present due to the tremendous size of the Bitcoin transaction history and the continuous growth of the Bitcoin blockchain. Additionally, the system must effectively and correctly present probability results in a timely manner to ensure users understand the limitations of risk-based analysis.

5. Testing

The frontend of the system will be implemented using Blazor and developed and tested within the Visual Studio integrated development environment. The frontend testing will combine automated component testing with end-to-end testing to ensure correct input validation, accurate rendering of probability results, and reliable user interaction with the backend computation service.

Automated component testing will be written using bUnit and xUnit and executed through Visual Studio's built-in testing and debugging tools. These tests verify that user inputs are properly validated, that the interface transitions correctly between idle, loading, and result states, and that the probability values returned from the backend are rendered accurately. The

backend API calls will be mocked to allow frontend behavior to be tested independently of the backend availability.

Manual end-to-end testing will also be performed to evaluate the complete user workflow, which includes parameter entry, submission, error handling, and result display. These tests confirm that frontend components integrate correctly with backend services and that the application behaves predictably under normal and invalid usage scenarios. This combined testing approach provided confidence that the Blazor frontend operates correctly and reliably as part of the overall system.

Backend component testing will involve using the Postman API testing tools to perform unit testing on individual endpoints to ensure correct responses for a given request, as well as end-to-end testing to ensure multiple endpoints will work together as expected. Testing the double-spend calculations will involve manual testing with known values to ensure that the probabilities are being calculated correctly.

Integration testing will be conducted to verify correct interactions between the frontend and backend probability computation service. These tests ensure that user inputs are correctly transmitted to the backend, that responses are returned in the expected format, and that the probability results are accurately displayed in the frontend. The testing will be performed in a controlled environment using Visual Studio, with a running backend service. Integration testing confirms that independently tested frontend and backend components function correctly when combined and that error responses are handled appropriately.

6. Planning

Milestone 1: Requirements & Design - Week 1

- Define the project scope, system requirements, and overall system architecture.
- Plan the implementation strategy for the backend, frontend, and Bitcoin Core node.
- Determine how system components will interact and communicate with one another.
- Select development tools and technologies, including:
    - Blazor and MudBlazor for the frontend user interface
    - Bitcoin Core for blockchain data access
    - C# and Python for backend services and processing

Milestone 2: Implementation of Bitcoin Node - Week 2

- Set up Bitcoin core on a Linux server
- Sync to the blockchain
- Test the communication function

Milestone 3: Integrate Double-Spend Probability Calculations

- Double-spend formulas implemented and tested
- Integrate with the back end

Milestone 4: Backend API / Frontend Demo

- Implement backend API
- Frontend UI Demo complete

Milestone 5: Frontend Integration with API

- Frontend UI and Backend API integrated and tested for proper functionality

Milestone 6: Testing, Demo, Finalize

- Finalize implementation of features
- Have a working demo tested

6.1 Workstreams:

System architecture, frontend/backend integration, Bitcoin-node setup, and testing.

6.2 Deliverables:

The primary deliverable of this project is a user-friendly homepage that allows users to enter a Bitcoin transaction ID and analyze the probability of that transaction being successfully double-spent. The homepage will display and elaborate upon essential information explaining how a double-spend attack occurs, why it is an important security concern within the Bitcoin blockchain, and how transaction confirmations and time reduce the likelihood of fraud. The website will also display and explain mathematical equations used to calculate the Bitcoin double-spend probability. The equations are derived from the Bitcoin

whitepaper [1] and the acceptance-policy analysis in [2].

The homepage will have user-friendly instructions on how to use the webpage by entering their transaction ID and how they can interpret the results.

Users will be able to search for a transaction ID for both historical transactions and unconfirmed transactions that are currently present within the mempool.

References

[1] "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] "An analysis of acceptance policies for blockchain transactions," IACR Cryptology ePrint Archive, Rep. 2018/040, Jan. 2018. [Online]. Available: https://eprint.iacr.org/2018/040

[3] "The Clean Architecture," Aug. 13, 2012. [Online]. Available: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html